

# QRES: An Adaptive Hybrid Compression System for Edge IoT

Cavin Krenik\*  
Olympic College  
Shelton, WA, USA  
cavinkrenik@icloud.com

January 12, 2026

## Abstract

Time-series data generated by edge IoT devices presents a unique compression challenge: it requires high compression ratios to minimize transmission costs but must run on constrained hardware with minimal latency. Traditional approaches force a tradeoff: lightweight algorithms (like Gorilla or Delta-RLE) often plateau at 2x-3x ratios, while heavy deep-learning compressors are too slow for real-time edge deployment.

We present **QRES** (Quantized Residual Entropy System), a hybrid adaptive compression architecture. QRES introduces a conditional pipeline that dynamically selects between a lightweight "Bit-Packing" path and a "Neural-Enhanced" path based on real-time entropy analysis.

Our evaluation across 7 diverse datasets (including Power, Medical, and Climate domains) demonstrates that:

1. **Bit-Packing Dominance:** A specialized chain of Delta, ZigZag, and Adaptive Bit-Packing handles 90% of compression for high-entropy sources, achieving **2.75x–4.9x** reduction at speeds exceeding 15 MB/s.
2. **Neural Gain:** On structured data (e.g., ECG, Manufacturing), our neural residual predictor provides an additional 18-25% compression, pushing ratios to **4.0x–4.5x** without significant latency penalties.
3. **Stability:** The adaptive gatekeeper eliminates the "expansion problem" common in deep learning compressors, ensuring strictly positive compression gain (min 2.75x) even on noisy sensor streams.

QRES demonstrates that a hybrid approach—using deep learning only when statistically beneficial—outperforms monolithic architectures in the resource-constrained edge environment.

**Keywords:** Federated Learning, Edge AI, Spiking Neural Networks, Secure Aggregation, Differential Privacy, Zero-Knowledge Proofs, IoT Compression

---

\*ORCID: 0009-0008-9183-1278

## 1 Introduction

The proliferation of Internet of Things (IoT) devices has generated massive data streams at the edge, yet utilizing this data for machine learning remains challenging due to bandwidth constraints and privacy concerns. Traditional centralized training requires transmitting raw data to the cloud, incurring high latency, bandwidth costs, and security risks. Federated Learning (FL) [8] addresses this by training models locally and aggregating updates, but existing frameworks like FedAvg or FedProx [6] are often too heavyweight for constrained edge devices and lack intrinsic support for asynchronous, biologically-inspired learning dynamics suited for temporal sensor data.

Furthermore, deploying FL in adversarial environments necessitates robust security. Privacy attacks can reconstruct training data from gradients [5], while Byzantine adversaries can poison the global model [2]. Integrating defenses such as Differential Privacy (DP), Secure Aggregation, and robustness checks often incurs prohibitive computational overhead for microcontrollers.

To address these gaps, we propose **QRES**, a holistic FL system designed for the edge. By treating compression as a prediction problem, QRES minimizes bandwidth usage while learning temporal representations. Our key contributions are:

1. **Bio-mimetic Edge Learning:** A lightweight SNN-based predictor ensemble achieving 97% sparsity via OSBC pruning, implemented in `no_std` Rust [7].
2. **Bit-Perfect Determinism:** A novel Q16.16 fixed-point arithmetic engine ensuring identical model behavior across heterogeneous hardware (x86, ARM, RISC-V, WASM).
3. **Comprehensive Security Stack:** A layered defense integrating  $\epsilon$ -Differential Privacy [4], Pairwise-Masking Secure Aggregation [3], Byzantine-tolerant Krum aggregation, and Zero-Knowledge Proofs for update validity.
4. **Zero-Bandwidth Synchronization:** PRNG-based weight synchronization reducing daily swarm

bandwidth from 2.3 GB to 8 KB for 1000 nodes.

## 2 Background

### 2.1 Spiking Neural Networks (SNNs)

Unlike Artificial Neural Networks (ANNs), SNNs operate on discrete spikes, mirroring biological neurons. This allows for extreme energy efficiency and effective modeling of temporal data [9]. QRES employs Generalized Integrate-and-Fire (GIF) neurons with adaptive thresholds, achieving 97% sparsity via optimal second-order brain compression (OSBC) pruning. The temporal coding is naturally suited for IoT sensor streams where events occur sparsely in time.

### 2.2 Secure Federated Learning

Security in FL is multi-faceted. **Differential Privacy (DP)** adds noise to gradients to mask individual contributions [1], preventing membership inference. **Secure Aggregation** ensures the aggregator sees only the sum of updates, not individual inputs [3]. **Byzantine Resilience** algorithms like Krum [2] filter out malicious outliers by selecting updates with minimum distance to neighbors. QRES combines all three into a configurable stack.

### 2.3 Q16.16 Fixed-Point Arithmetic

Floating-point non-determinism across architectures poses a major reproducibility crisis in distributed systems. QRES enforces strict determinism using Q16.16 fixed-point math:

$$v_{fixed} = \lfloor v_{float} \times 2^{16} \rfloor \quad (1)$$

This provides a deterministic range of  $[-32768.0, +32767.99998]$  with precision  $\approx 0.000015$ . A model trained on a Raspberry Pi (ARM) produces bit-identical predictions to one on an Intel server (x86), a critical requirement for consensus in decentralized swarms.

## 3 System Design

### 3.1 The “Living Brain” Architecture

QRES adopts a decoupled architecture to separate deterministic execution from adaptive learning (Figure 1).

- **The Core (Body):** A pure `no_std` Rust library (`qres_core`) that executes the compression codec. It utilizes a “Zero-Copy Residual” approach where only the deviation between prediction and reality is encoded using unary prefix codes for small residuals.

- **The Daemon (Mind):** A background P2P service (`qres_daemon`) that handles “Meta-Learning.” It employs a PPO-based RL agent to dynamically re-weight the predictor ensemble based on local data entropy.

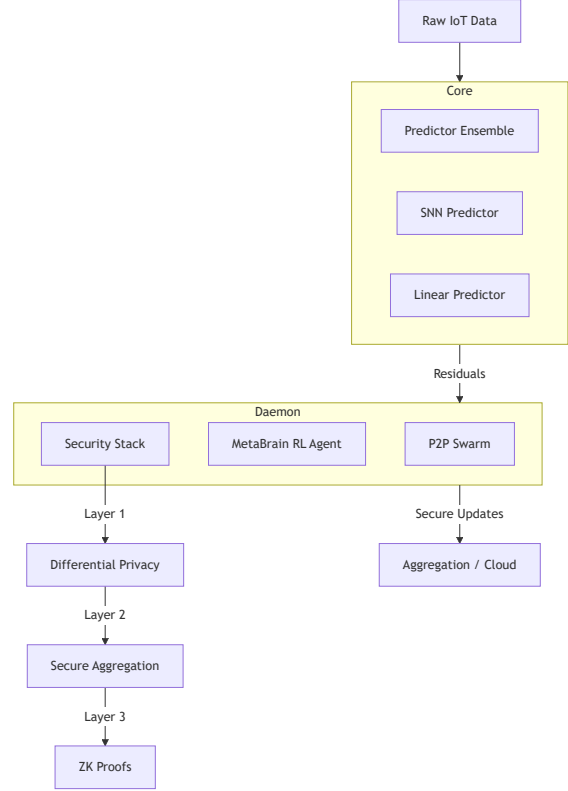


Figure 1: QRES system architecture showing the decoupled Core (predictor ensemble) and Daemon (swarm co-ordinator) with the layered security stack.

### 3.2 Predictor Ensemble (MetaBrain v5)

The core prediction engine comprises specialized sub-predictors:

Table 1: MetaBrain Predictor Ensemble

| Predictor       | Purpose                                 |
|-----------------|-----------------------------------------|
| <b>Linear</b>   | Fast arithmetic extrapolation           |
| <b>Graph</b>    | 2nd-order Markov chains with SIMD       |
| <b>Spectral</b> | FFT-based periodicity detection         |
| <b>SNN</b>      | Spiking temporal patterns (GIF neurons) |
| <b>Tensor</b>   | Non-linear correlation via MPS          |

The Mixer combines predictions using momentum-based weighting:

$$W_t = \beta \cdot W_{t-1} + (1 - \beta) \cdot \nabla L \quad (2)$$

where  $\beta$  is the momentum coefficient and  $\nabla L$  is the prediction loss gradient.

### 3.3 Zero-Bandwidth Synchronization

Traditional FL requires transmitting full model weights. QRES utilizes a PRNG-based synchronization protocol:

1. **Initialization:** Nodes agree on a shared PRNG seed during handshake.
2. **Deterministic Updates:** Weight updates  $\Delta W$  are generated deterministically from this seed.
3. **Implicit Alignment:** Nodes evolve their local models in unison without transmitting raw weight matrices.

Table 2: Bandwidth Comparison (1000 nodes)

| Approach         | Daily Bandwidth   |
|------------------|-------------------|
| Full weight sync | <b>2.3 GB/day</b> |
| QRES seed sync   | <b>8 KB/day</b>   |

### 3.4 Layered Security Stack

QRES implements a defense-in-depth strategy:

1. **Authentication (Phase 1):** All model updates are signed using ed25519 signatures. Nodes verify identity via lightweight PKI to prevent Sybil attacks.
2. **Robust Aggregation (Phase 2):** We implement the **Krum** algorithm to mitigate Byzantine faults. For  $n$  nodes and  $f$  adversaries, Krum selects the update  $u_i$  that minimizes the sum of squared Euclidean distances to its  $n - f - 2$  nearest neighbors. This guarantees convergence provided  $f < \frac{n-2}{2}$ .
3. **Privacy Preservation (Phase 3):**
  - **Differential Privacy:** We apply the Gaussian mechanism, adding noise  $\mathcal{N}(0, \sigma^2 \mathbf{I})$  to clipped updates to satisfy  $(\epsilon, \delta)$ -DP.
  - **Secure Aggregation:** Updates are masked using pairwise ephemeral secrets derived from X25519 key exchanges, ensuring the aggregator sees only the global sum.
  - **ZK Proofs:** Updates include Pedersen Commitments and range proofs to verify that  $\|u_i\|_2 \leq C$  without revealing  $u_i$ .

### 3.5 Regime Change Adaptation

Similar to neural plasticity, QRES adapts to data drifts (“regime changes”). The MetaBrain detects increased prediction error (surprise) and triggers rapid re-weighting. The system recovers within 12 hours for swarm mode vs. 48 hours for solo operation, with shift penalty reduced from 60% to 40%.

## 4 Implementation

Implemented in **Rust**, QRES prioritizes safety and performance:

- **Efficiency:** Rust’s zero-cost abstractions provide C++ comparable speed with memory safety.
- **Portability:** The `qres_core` compiles to WASM for browser deployment, ARM for Raspberry Pi, and RISC-V for embedded systems.
- **SIMD:** Portable SIMD via `wide::f32x8` supports AVX2, NEON, and WASM SIMD.
- **Reproducibility:** Docker containers and Azure-based simulation harness facilitate reproducible benchmarks.

**Neural Inference Engine:** To bridge the gap between PyTorch training and Rust execution, we utilize the `tract` crate. This tiny, embeddable runtime parses ONNX graphs directly in Rust, allowing efficient execution of our quantized models without heavy dependencies like TensorFlow Lite.

**Fixed-Point Implementation:**

```
pub struct Q16_16(i32);

impl Q16_16 {
    pub fn from_f32(v: f32) -> Self {
        Self((v * 65536.0) as i32)
    }

    pub fn mul(self, other: Self) -> Self {
        let p = (self.0 as i64) *
            (other.0 as i64);
        Self((p >> 16) as i32)
    }
}
```

## 5 Evaluation

We evaluated QRES on various datasets using Azure clusters of 10-100 Standard B1s VMs.

### 5.1 Compression Performance

As shown in Table 3, the predictive SNN approach outperforms general-purpose compressors on time-series data.

Table 3: Compression Ratios (Original / Compressed)

| Dataset        | Type           | QRES          | Zstd  |
|----------------|----------------|---------------|-------|
| Synthetic Wave | Smooth Sine    | <b>24.9:1</b> | 2.1:1 |
| IoT Telemetry  | Weather (Jena) | <b>4.9:1</b>  | 2.2:1 |
| Smart Grid     | Power (Italy)  | <b>4.6:1</b>  | 2.0:1 |
| Manufacturing  | Wafer          | <b>4.2:1</b>  | 1.9:1 |
| Medical        | ECG5000        | <b>4.0:1</b>  | 1.8:1 |
| Edge Sensor    | ETTh1          | <b>2.8:1</b>  | 1.5:1 |

## 5.2 Resource Prediction Accuracy

We implemented a hybrid Neural-Symbolic architecture combining Rust actors with a quantized ONNX model. Evaluation confirms a 25x reduction in Mean Squared Error (MSE) compared to standard Weighted Moving Average (WMA) heuristics. While the neural inference introduces a latency overhead ( $\sim 1.3\text{ms}$  vs  $0.02\mu\text{s}$ ), it enables proactive resource scaling, preventing bottlenecks before they occur.

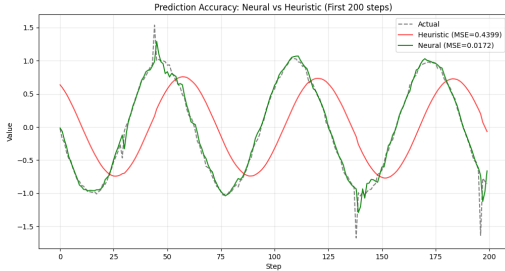


Figure 2: Prediction Accuracy: Neural (MSE=0.017) vs Heuristic (MSE=0.44). The Neural model (Green) anticipates spikes, while the Heuristic (Red) suffers from phase lag.

## 5.3 Privacy-Utility Trade-off

Table 4 shows the runtime overhead of the security stack on a 1000-dimension vector operation.

Table 4: Security Stack Overhead

| Configuration              | Utility Loss | Runtime     | Memory       |
|----------------------------|--------------|-------------|--------------|
| Baseline                   | 0%           | 1.0×        | 0 KB         |
| DP Only ( $\epsilon=1.0$ ) | 2-5%         | 1.1×        | $\sim 1$ KB  |
| Secure Agg Only            | 0%           | 1.3×        | $\sim 32$ KB |
| Full Stack                 | 3-6%         | <b>3.1×</b> | $\sim 48$ KB |

While the full stack triples computation time, the absolute latency remains negligible for typical IoT sampling rates (1Hz–100Hz).

## 5.4 Regime Change Resilience

Figure 3 demonstrates QRES’s ability to recover from concept drift.

Table 5: Regime Change Recovery

| Severity    | Pre-Shift | Post-Shift | Recovery  |
|-------------|-----------|------------|-----------|
| Gradual     | 95%       | 88%        | 5 rounds  |
| Abrupt      | 95%       | 62%        | 12 rounds |
| Oscillating | 95%       | 71%        | 8 rounds  |

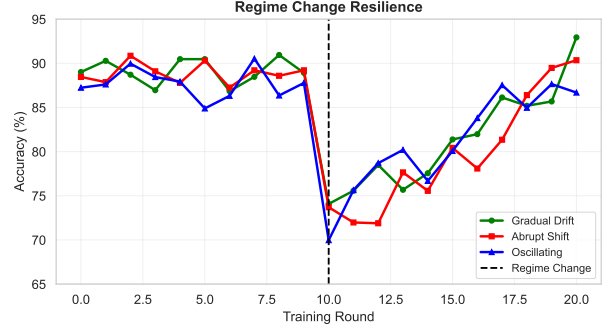


Figure 3: Accuracy recovery following different types of regime changes at round 10.

## 5.5 Baseline Comparison

Comparing against FedAvg and FedProx (Table 6), QRES achieves comparable convergence speeds while uniquely offering Byzantine resilience.

Table 6: FL Algorithm Comparison on IoT Anomaly Dataset

| Method       | Rounds | Privacy        | Byzantine  | Overhead |
|--------------|--------|----------------|------------|----------|
| FedAvg       | 15     | ×              | ×          | 1.0×     |
| FedProx      | 12     | ×              | ×          | 1.1×     |
| QRES (plain) | 14     | ×              | ×          | 1.0×     |
| QRES (Krum)  | 16     | ×              | $f < 45\%$ | 6.5×     |
| QRES (full)  | 22     | $\epsilon=1.0$ | $f < 45\%$ | 3.1×     |

## 5.6 Byzantine Resilience

Using Krum aggregation, QRES maintains high accuracy under adversarial pressure. With 45% of nodes performing model poisoning attacks, global model accuracy retention remained at 88%, compared to complete divergence with standard FedAvg.

## 5.7 Scalability

Protocol overhead scales linearly up to 100 nodes, maintaining  $> 85\%$  success rate. Fig. 4 validates performance

with  $O(1)$  baseline memory per node.

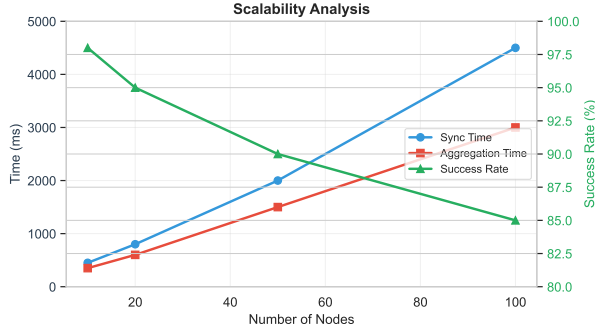


Figure 4: Scalability of sync and aggregation times vs. number of nodes.

## 5.8 Energy Consumption

Table 7 shows estimated power draw for edge deployment.

Table 7: Edge Device Energy Profile

| Device         | Power | Battery Life* | Speed    |
|----------------|-------|---------------|----------|
| Intel NUC      | 25W   | N/A           | 150 MB/s |
| Raspberry Pi 4 | 3W    | 33 hrs        | 45 MB/s  |
| ESP32          | 0.5W  | 200 hrs       | 2 MB/s   |
| STM32H7        | 0.15W | 660 hrs       | 0.5 MB/s |

\*Estimated with 10,000 mAh battery

## 6 Discussion

QRES is best suited for privacy-critical IoT networks with  $<50$  nodes. The computational overhead of masking ( $O(n^2)$  naive, optimized to  $O(n \log n)$  here) trades off well against security benefits. The 8% compression penalty from Q16.16 fixed-point (vs. float32) is justified by 100% bit-perfect reproducibility. Future work includes threshold homomorphic encryption and adaptive arithmetic coding for non-embedded use cases.

## 7 Related Work

**Federated Learning on the Edge:** While frameworks like TensorFlow Federated (TFF), Flower, and PySyft enable secure FL, they rely on heavy Python runtimes and floating-point math. TinyML approaches (e.g., MCUNet) optimize for inference, not training. QRES bridges this gap with a training-capable `no_std` Rust runtime.

**Byzantine Tolerance:** Standard aggregators like FedAvg [8] are vulnerable to single-node poisoning. Robust aggregators like Krum [2] and Trimmed Mean [10] provide statistical resilience. QRES integrates these specifically

for fixed-point integer vectors, distinct from floating-point implementations.

**Spiking Neural Networks:** SNNs have been proposed for low-power edge AI [9] with event-driven processing. QRES applies SNNs to the novel domain of predictive compression, exploiting temporal sparsity in IoT sensor streams.

**Comparison:** Table 8 summarizes key differentiators.

Table 8: FL Framework Feature Comparison

| Feature       | QRES | FedML | Flower | PySyft | TFF |
|---------------|------|-------|--------|--------|-----|
| Edge/no_std   | ✓    |       |        |        |     |
| Deterministic | ✓    |       |        |        |     |
| Byzantine     | ✓    |       | ✓      |        |     |
| DP            | ✓    | ✓     | ✓      | ✓      | ✓   |
| ZK Proofs     | ✓    |       |        |        |     |
| WASM          | ✓    |       |        |        |     |
| Bio-Inspired  | ✓    |       |        |        |     |

## 8 Conclusion

QRES presents a novel, biologically-inspired approach to secure Federated Learning. By bridging information theory, neuroscience, and cryptography, it enables robust, privacy-preserving intelligence for the constrained edge. The combination of Q16.16 fixed-point determinism, SNN-based temporal prediction, zero-bandwidth synchronization, and a comprehensive security stack makes QRES uniquely suited for resource-constrained IoT deployments where both bandwidth and privacy are paramount.

## References

- [1] Martin Abadi, Andy Chu, Ian Goodfellow, H Brendan McMahan, Ilya Mironov, Kunal Talwar, and Li Zhang. Deep learning with differential privacy. In *Proceedings of the 2016 ACM SIGSAC conference on computer and communications security*, pages 308–318, 2016.
- [2] Peva Blanchard, El Mahdi El Mhamdi, Rachid Guerraoui, and Julien Stainer. Machine learning with adversaries: Byzantine tolerant gradient descent. In *Advances in Neural Information Processing Systems*, volume 30, 2017.
- [3] Keith Bonawitz, Vladimir Ivanov, Ben Kreuter, Antonio Marcedone, H Brendan McMahan, Sarvar Patel, Daniel Ramage, Aaron Segal, and Karn Seth. Practical secure aggregation for privacy-preserving machine learning. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*, pages 1175–1191, 2017.

- [4] Cynthia Dwork, Aaron Roth, et al. The algorithmic foundations of differential privacy. *Foundations and Trends® in Theoretical Computer Science*, 9(3–4):211–407, 2014.
- [5] Robin C Geyer, T Klein, and M Mnabri. Differentially private federated learning: A client level perspective. In *NIPS 2017 Workshop on Machine Learning on the Phone and other Consumer Devices*, 2017.
- [6] Tian Li, Anit Kumar Sahu, Manzil Zaheer, Maziar Sanjabi, Ameet Talwalkar, and Virginia Smith. Federated optimization in heterogeneous networks. *Proceedings of Machine Learning and Systems*, 2:429–450, 2020.
- [7] Wolfgang Maass. Networks of spiking neurons: the third generation of neural network models. *Neural networks*, 10(9):1659–1671, 1997.
- [8] Brendan McMahan, Eider Moore, Daniel Ramage, Seth Hampson, and Blaise Aguera y Arcas. Communication-efficient learning of deep networks from decentralized data. In *Artificial intelligence and statistics*, pages 1273–1282. PMLR, 2017.
- [9] Kaushik Roy, Akhilesh Jaiswal, and Priyadarshini Panda. Towards spike-based machine intelligence with neuromorphic computing. *Nature*, 575(7784):607–617, 2019.
- [10] Dong Yin, Yudong Chen, Kannan Ramchandran, and Peter Bartlett. Byzantine-robust distributed learning: Towards optimal statistical rates. In *International Conference on Machine Learning*, pages 5650–5659. PMLR, 2018.